

Sources

<https://www.geeksforgeeks.org/engineering-mathematics/graph-measurements-length-distance-diameter-eccentricity-radius-center/>

<https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>

1

Let $e(a)$ be the eccentricity of node a . We can write $diam(G) = \max_{v \in V(G)} [e(v)]$ and $rad(G) = \min_{v \in V(G)} [e(v)]$

Proving $rad(G) \leq diam(G) \leq 2rad(G)$

Trivially true for a disconnected graph because $\forall v \in V(G), e(v) = \infty$ so $\infty \leq \infty \leq 2\infty$ which is true.

Now from our above rewrite, $\min_{v \in V(G)} [e(v)] \leq \max_{v \in V(G)} [e(v)]$ which is trivially true. The harder part of the proof is $\max_{v \in V(G)} [e(v)] \leq 2 \min_{v \in V(G)} [e(v)]$

Proof by contradiction, assume $\max_{v \in V(G)} [e(v)] > 2 \min_{v \in V(G)} [e(v)]$, which means there are vertices i, j with longest paths from these nodes being d, r such that $d > 2r$.

Now we know that node j cannot be in the path of node i that results in a distance d otherwise we would either trivially make a new path for j that is greater than r (contradiction because r is j 's longest path) or the case for j is in the middle of the path then $2r$ would be greater than or equal to d which is another contradiction.

But we know that j must connect to i because if it cannot then we fall into disconnected graphs which were explained above. If j connects to i using nodes not in the longest path then we can create a longer path and thus a contradiction, so j must connect to i using nodes in the path. This follows the same logic as a previous hw / the paragraph above where j to i or j to the end of the longest path must be a distance r . The reason why j 's path must end at either i or the other side of i 's longest path because if it doesn't then we can easily construct a longer path (because we know at least one vertex in j 's path intersects with i 's longest path). This means that r is now at least half of d this creates a contradiction because r should be less than half of d from our assumption.

2

Prim's algorithm essentially works by partitioning the graph into visited cut and taking the minimum edge across the cut and adding this new vertex to our visited cut. Suppose we treat $E(P)$ like an ordered set of edges that are ordered in when they were added from Prim's algorithm.

If $T = P$ yay were done, otherwise $\exists e, e \in E(P) \wedge e \notin E(T)$ and let this e be the first edge in $E(P)$ where this is true. Now if we remove e from P , we now have a graph split into two parts, this is our new cut C . Since T is a spanning tree and we know C is the same for T and P because the edges being ordered and since $V(T) = V(G) = V(P)$ then $V(P) - C = V(T) - C$. Now since T is a MST, there must be an edge f that crosses the cut in T . We replace f in T with e to create T' which would not increase the total weight because since e was chosen by Prim's algorithm instead of f , then $w(e) \leq w(f)$. Thus, $W(T') \leq W(T)$. Now we check if T' is still a spanning tree and it is because $T - f$ has two components (which are C and $T - C$) and adding e bridges the two components so there is no cycle unless $T - f$ had only one component in which case T would not be a spanning tree (so contradiction).

Thus we repeat the above steps until $P = T'$ and at each replacement step we either decreased the total weight of the tree or kept it the same and preserved the fact that we have a tree. Since we were given that T is a minimum spanning tree then we could not decrease the total weight and instead find that we are maintaining the total weight of the tree and that $W(T) = W(P)$ so P is a MST.

3

Dijkstra's algorithm always moves to the next closest node from the source that was not visited. The basic assumption is that we're always on the shortest path between the source and our target node, thus we can always look at the distance from a node to the source as $p + w(e)$ because we assume we are on the shortest path. With the existence of a negative edge, this assumption may no longer hold because if we can take the negative edge back to a visited node u at a shorter distance then we no longer are always taking the shortest paths.

Take for example a graph with nodes s as source, i, j with an edge whose weight is $w(i, j) = -99$. Suppose $w(s, j) = 5$ and the shortest path to i from s is 100. When we eventually get to i , Dijkstra would not "take" the edge (i, j) because we have already visited j even though we should take this edge as it results in a shorter distance to j than the distance of 10 that we marked.

4

No negative weight cycle simply implies that we cannot get back to the same vertex with a negative distance traveled. While Dijkstra's algorithm could fail here, we can use the same principles in our proof.

Proof by contradiction

Let $p_s(v)$ be the edges used in the shortest path from node s to v . Suppose $T = \cup_{v \in V(G)} p_s(v)$ does not result in a spanning tree.

Well we know that $V(T) = V(G)$ from the construction and that $|E(T)| \geq |V(G)| - 1$ but since we are assuming that T is not a spanning tree then there must be at least one extra edge from say $p_s(i)$. This extra edge would create a cycle somewhere in the graph. If there is a cycle, then there would be 2 different ways to get to a single vertex j (one path that uses the extra edge and one that used the edges from $p_s(j)$). We know that the shortest path from s to j is by using $p_s(j)$ so if $p_s(i)$ creates this cycle then j must be in $p_s(i)$ in which case altering the subpath of $p_s(i)$ to take the shortest path for j would result in a shorter path for i which creates a contradiction.

It is important to note that there is an edge case where the two paths to our node j in the example are of equal length, in which case removing one of them would result in the spanning tree while still keeping the shortest path constraint true. Therefore we are able to construct a T of shortest paths that is a spanning tree and maintains the shortest distances from s to all other nodes.